J.U.C = java.util.concurrent

1 ReentrantLock（重入锁）

通过代码演示 ReentrantLock 锁的作用（thread-juc 项目下的 AtomicDemo）

```
public class AtomicDemo {
    private static int count = 0;
    static Lock lock = new ReentrantLock();
    public static void inc() {
        lock.lock();
        try {
            Thread.sleep(1);
            count++;
        } catch (InterruptedException e) {
            e.printStackTrace();
        }finally {
            lock.unlock();
        }
    }

    public static void dec() {
        lock.lock();
        try {
            Thread.sleep(1);
            count--;
        } catch (InterruptedException e) {
            e.printStackTrace();
        }finally {
            lock.unlock();
        }
    }

    public static void main(String[] args) throws InterruptedException
{
        for(int i=0;i<1000;i++){
            new Thread(()->{AtomicDemo.inc();}).start();
        }
        Thread.sleep(4000);
        System.out.println(count);
    }
}
```

比 synchronized 的优势在于：

1）效率更高，表现在读读不互斥。

2）用法更灵活

2 ReentrantReadWriteLock（重入读写锁）

读读不互斥

读写互斥

写写互斥

通过代码演示读写锁互斥的概念（thread-juc 项目下的 ReadWriteDemo）

```
public class ReadWriteDemo {
    static Map<String,Object> cachMap = new HashMap<String,Object>();
    static ReentrantReadWriteLock rwl = new ReentrantReadWriteLock();
    static Lock read = rwl.readLock();
    static Lock write = rwl.writeLock();

    public static Object get(String key){
        read.lock();
        try {
            return cachMap.get(key);
        }finally {
            read.unlock();
        }
    }

    public static Object write(String key,Object value){
        write.lock();
        try {
            return cachMap.put(key,value);
        }finally {
            write.unlock();
        }
    }

    public static void main(String[] args) {

    }
}
```

ReentrantLock 和 ReentrantReadWriteLock 不存在对象锁和类锁这个概念，锁就取决于线程谁拿到了 lock。这个区别还是看代码（thread-juc 项目下的 AtomicDemo），当两个不同的方法中通过同一个 lock 上锁之后，如果线程 A 调 inc()，线程 B 调 dec(),这个时候锁是互斥的。不能同时调用

沸气的景天

3 重入锁

指的是在同一个 thread 中，获取锁之后再次使用同样的方法或对象中的其他方法可以直接操作，而不需要重新获取锁。它是基于 thread 粒度的，per-thread

```
public class RepeatLock {
    boolean isLocked = false;
    Thread lockedBy = null;
    int lockedCount = 0;
    public synchronized void lock()
            throws InterruptedException{
        Thread thread = Thread.currentThread();
        while(isLocked && lockedBy != thread){
            wait();
        }
        isLocked = true;
        lockedCount++;
        lockedBy = thread;
    }
    public synchronized void unlock(){
        if(Thread.currentThread() == this.lockedBy){
            lockedCount--;
            if(lockedCount == 0){
                isLocked = false;
                notify();
            }
        }
    }
}

public class Count {
    // 不可重入锁
    UnRepeatLock lock = new UnRepeatLock();
    public void print() {
        System.out.println("获取 lock");
        lock.lock();
        System.out.println("调用 doAdd 之前");
        doAdd();
        System.out.println("调用 doAdd 之后");
        lock.unlock();
        System.out.println("释放 lock");
    }
    public void doAdd() {
        System.out.println("再次获取 lock");
        lock.lock();
        //do something
        lock.unlock();
        System.out.println("再次释放 lock");
    }
```

帅气的景天

```
    public static void main(String[] args) {
        Count cc = new Count();
        new Thread(()->{cc.print();}).start();
    }
}
```

4 不可重入锁

指的是每次使用锁方法时，都需要重新获取锁，即使在同一线程中调用同一方法都需要等待上一个锁的释放。它是基于 method 粒度的，per-invocation。

```
public class UnRepeatLock {
    private boolean isLocked = false;
    public synchronized void lock() {
        while(isLocked){
            try {
                wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        isLocked = true;
    }
    public synchronized void unlock(){
        isLocked = false;
        notify();
    }
}
```

帅气的景天

5 公平锁与非公平锁

公平锁：每个人正常排队正常竞争锁

非公平锁：新来的线程能抢走锁

reentrantlock 默认为非公平锁

6 思考锁的实现（设计思维）

实现锁的互斥特性

- 锁的互斥特性->要有共享资源->标记（0 无锁，1 有锁）
- 没有抢占到锁的线程？（等待->唤醒）
- 等待的线程怎么存储？->数据结构去存储一系列等待中的线程，FIFO

- 公平和非公平（能否插队）
- 重入的特性（识别是否是同一个线程）

技术方案

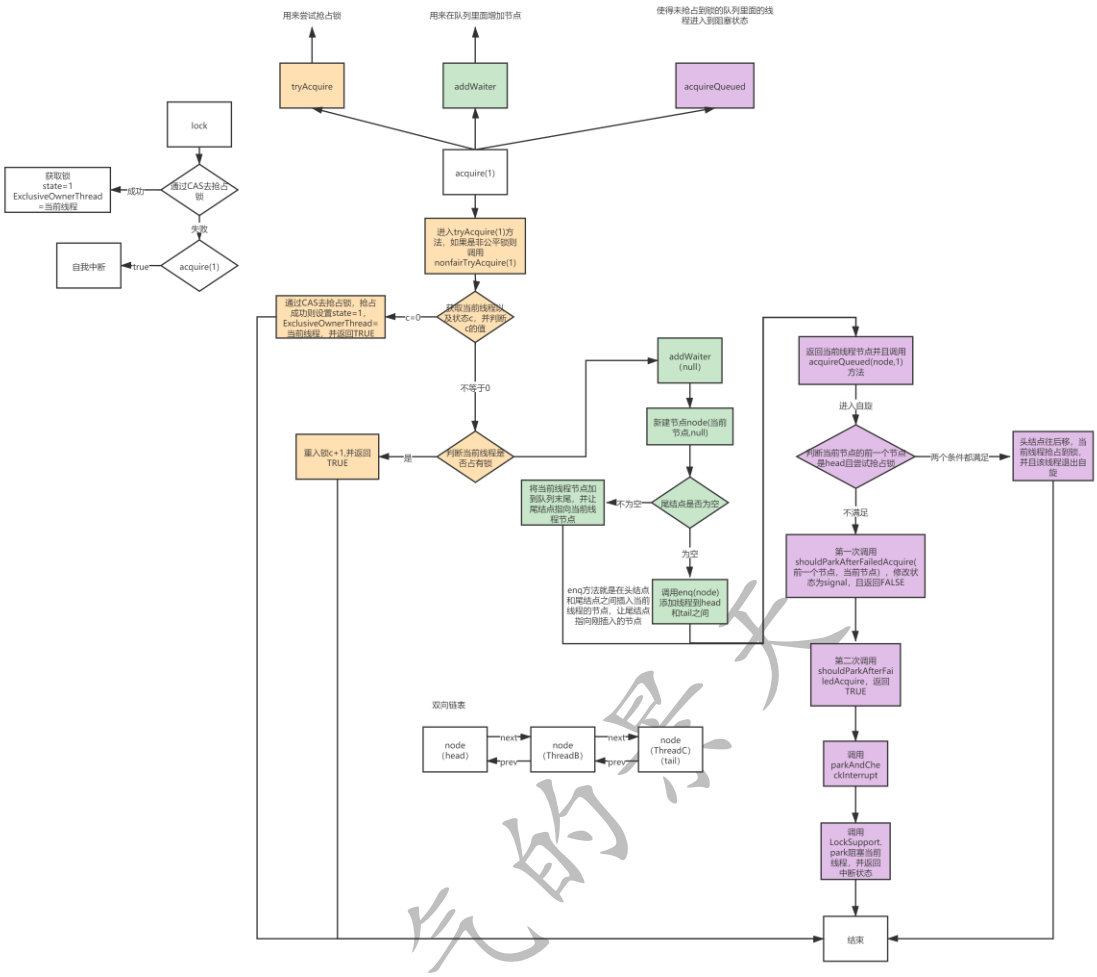
- volatile state=0（无锁），1 代表持有锁，>1 代表重入
- wait/notify | condition 无法实现唤醒指定线程->LockSupport.park()、unpark(thread)，是 unsafe 类提供的方法，可以传入指定线程，实现唤醒指定线程
- 双向链表
- 逻辑层面去实现
- 在某个地方存储当前获得锁的线程 ID，判断下次抢占锁的线程是否同一个。

---

7 源码解读

一边讲课一边画逻辑图

帅气的景天



7.1 非公平锁实现

线程 A 进来，调用 lock 方法

```
final void lock() {
    //抢占互斥资源
    //if(state==0){
        //然后再去修改状态
    //}
    //上面这种方式肯定不行，因为可能同时有多个线程进入到 if 语句，这就意味着多个线程抢占到同一把锁
    //不管当前队列是否有人排队，所有锁都可以去抢
    if (compareAndSetState(0, 1))//通过 CAS 去抢（乐观锁）
        //保存当前线程
        setExclusiveOwnerThread(Thread.currentThread());
    else
```

```
        acquire(1);
}
```

1.线程 A 尝试去拿到锁，这个时候 state=0,能够拿到锁，把 state 改为 1，调用 setExclusiveOwnerThread 方法，赋值 exclusiveOwnerThread=线程 A

2.线程 B 进来，尝试去拿锁，拿不到锁，进入 acquire(1)方法

```
public final void acquire(int arg) {
    if (!tryAcquire(arg) &&
        acquireQueued(addWaiter(Node.EXCLUSIVE), arg))
        selfInterrupt();
}
```

主要有 3 个方法，

tryAcquire

addWaiter

acquireQueued

我们先看第一个 tryAcquire(arg)，最终进入非公平锁实现类

```
  protected final boolean tryAcquire(int acquires) {
      return nonfairTryAcquire(acquires);
  }
```

```
final boolean nonfairTryAcquire(int acquires) {
    //得到当前线程
    final Thread current = Thread.currentThread();
    int c = getState();
    //如果是0，说明线程A 已经释放锁，现在我们假如的是A 还没有释放，所以不为0，如果已经执行完，，按照刚才的逻辑把state 改为1，调用setExclusiveOwnerThread 方法赋值exclusiveOwnerThread=线程B
    if (c == 0) {
        if (compareAndSetState(0, acquires)) {
            setExclusiveOwnerThread(current);
            return true;
        }
    }
    //如果当前线程是已经保存的exclusiveOwnerThread，表明同一个线程重入，只需要加重入次数
    else if (current == getExclusiveOwnerThread()) {
        int nextc = c + acquires;
        if (nextc < 0) // overflow
            throw new Error("Maximum lock count exceeded");
```

```
        setState(nextc);//这里不需要CAS 是因为当前线程已经获得锁了
        //为什么要统计累加次数，是为了匹配lock 和unlock 成对出现
        return true;
    }
    return false;
}
```

按照我们的假设，线程 B 都不满足，nonfairTryAcquire 返回 false,继续执行 acquireQueued(addWaiter(Node.EXCLUSIVE), arg))方法，我们先看 addWaiter

```
//mode 传入参数为null
private Node addWaiter(Node mode) {
    //node 的构造函数，赋值node 的nextWaiter 与thread，nextWaiter=null th
read=ThreadB  this.nextWaiter = mode;this.thread = thread;
    Node node = new Node(Thread.currentThread(), mode);
    // Try the fast path of enq; backup to full enq on failure
    //tail 默认为null，链表的尾结点
    Node pred = tail;
    if (pred != null) {
        node.prev = pred;
        if (compareAndSetTail(pred, node)) {
            pred.next = node;
            return node;
        }
    }
    enq(node);
    return node;
}
```

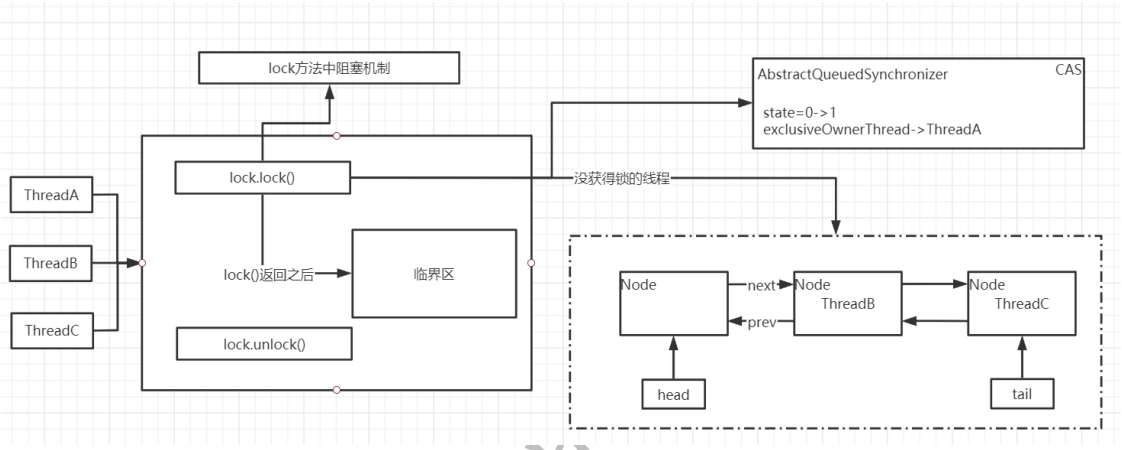
pred=tail 为 null ,进入 enq 方法

```
private Node enq(final Node node) {
    //跟while(1)的区别，while 从反汇编来看，代码更长，时间更久，没啥区别
    for (;;) {
        //tail 为null
        Node t = tail;
        if (t == null) { // Must initialize
            //尝试设置head=new node
            if (compareAndSetHead(new Node()))
                //将tail 设置为head, tail  head 都为new    Node()
                //再进入循环，t=new Node().进入else
                tail = head;
        } else {
            //t=new node  ,node 为刚才ThreadB 的node
            node.prev = t;
            //尝试把ThreadB 的node 赋值给tail
            if (compareAndSetTail(t, node)) {
```

帅气的景天

```
                //赋值成功，headnode 的后节点指向B 的node,tail=B 线程节点
                t.next = node;
                //返回head 节点,跳出方法
                return t;
            }
        }
    }
}
```

到这，形成了一个链表 node,并且返回 tail 节点



继续调用

```
//node 为ThreadB 的node,也就是tail 节点，arg=1
final boolean acquireQueued(final Node node, int arg) {
    boolean failed = true;
    try {
        boolean interrupted = false;
        for (;;) {
            //拿到ThreadB Node 的前指针节点
            final Node p = node.predecessor();
            //如果P==head 满足，并且尝试去拿锁，A 在占着，拿不到锁，走下一个
if
            if (p == head && tryAcquire(arg)) {
                //将Node 设置为head 节点，并且前后指针空指向，变成下图
                setHead(node);
                //把之前的head 的对象的后节点执行为空，让它进行GC 回收
                p.next = null; // help GC
                failed = false;
```

```
                return interrupted;
            }
            //A 没有释放，进入该逻辑，shouldParkAfterFailedAcquire 为false，
不执行，进入下次循环
            //B 还是拿不到锁，进入 shouldParkAfterFailedAcquire
            if (shouldParkAfterFailedAcquire(p, node) &&
                parkAndCheckInterrupt())
                interrupted = true;
        }
    } finally {
        if (failed)
            cancelAcquire(node);
    }
}
```

进入 shouldParkAfterFailedAcquire 方法，第一次进入把 head 节点的 waitStatus 改成 SIGNAL，第二次进入返回 true

```
//pred 为ThreadB 的Node 的前节点，即head 节点
private static boolean shouldParkAfterFailedAcquire(Node pred, Node nod
e) {
    // pred.waitStatus 默认为0,进入else 流程
    int ws = pred.waitStatus;
    if (ws == Node.SIGNAL)
        /*
         * This node has already set status asking a release
         * to signal it, so it can safely park.
         */
        return true;
    if (ws > 0) {
        /*
         * Predecessor was cancelled. Skip over predecessors and
         * indicate retry.
         */
        do {
            node.prev = pred = pred.prev;
        } while (pred.waitStatus > 0);
        pred.next = node;
    } else {
        /*
         * waitStatus must be 0 or PROPAGATE.  Indicate that we
         * need a signal, but don't park yet.  Caller will need to
         * retry to make sure it cannot acquire before parking.
         */
        //把head 的waitStatus 从0 改成SIGNAL
        compareAndSetWaitStatus(pred, ws, Node.SIGNAL);
    }
```

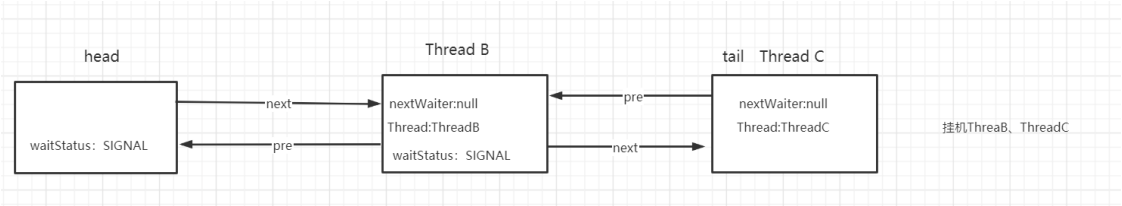
帅气的景天

```
    return false;
}
```

第二次进入 shouldParkAfterFailedAcquire 后返回 true，调用 parkAndCheckInterrupt 方法

```
private final boolean parkAndCheckInterrupt() {
    LockSupport.park(this); //挂机当前线程
    return Thread.interrupted();
}
```

同理，假如 A 没执行完，B 挂起，再来一个线程 C，最后会得到 B、C 都挂起



7.2 thread A unLock

```
//参数为1
public final boolean release(int arg) {
    //释放锁成功
    if (tryRelease(arg)) {
        Node h = head;
        if (h != null && h.waitStatus != 0)
            unparkSuccessor(h);
        return true;
    }
    return false;
}
```

//尝试去释放锁

```
//releases 传入为1
protected final boolean tryRelease(int releases) {
    //这一步state 为1,因为之前A 占着，得到c=0
    int c = getState() - releases;
    //判断是不是当前锁是A 占着
    if (Thread.currentThread() != getExclusiveOwnerThread())
```

```
        throw new IllegalMonitorStateException();
    boolean free = false;
    if (c == 0) {
        free = true;
        //释放锁,将exclusiveOwnerThread 设置为null
        setExclusiveOwnerThread(null);
    }
    //将state 设置为0
    setState(c);
    return free;
}
```

释放锁成功，之下如下逻辑

```
Node h = head;
//拿到head 的node 节点//首节点不为空，并且waitStatus 为SIGNAL -1
if (h != null && h.waitStatus != 0)
    //进入该逻辑
    unparkSuccessor(h);
return true;


//为一个new 的node 首节点
private void unparkSuccessor(Node node) {
    //ws 为-1
    int ws = node.waitStatus;
    if (ws < 0)
        //进入该逻辑，将head 节点的waitStatus 更改为0
        compareAndSetWaitStatus(node, ws, 0);

    //找到B 线程的node 节点
    Node s = node.next;
    //B 线程的node 节点不为空，waitStatus 为-1 不进入
    if (s == null || s.waitStatus > 0) {
        s = null;
        //如果B 节点失效，或者已经取消或者并发还没有赋值，从后往前找，找到第
一个需要执行的node,t==null 或者t==当前head 节点，才跳出
        //为什么？enq 方法里面，if 没有锁 线程A 通过CAS 进入if 语句块之后，发
生上下文切换，此时线程B 同样执行了该方法，并且执行完毕。然后线程C 调用了unpar
kSuccessor 方法。
        //假如是从头到尾的遍历形式，线程A 的next 指针此时还是null！也就是说，
会出现后续节点被漏掉的情况。
        for (Node t = tail; t != null && t != node; t = t.prev)
```

```
            if (t.waitStatus <= 0)
                s = t;
    }
    //得到我需要要unpark 的线程，这里为ThreadB
    if (s != null)
        //解锁threadB
        LockSupport.unpark(s.thread);
}
```

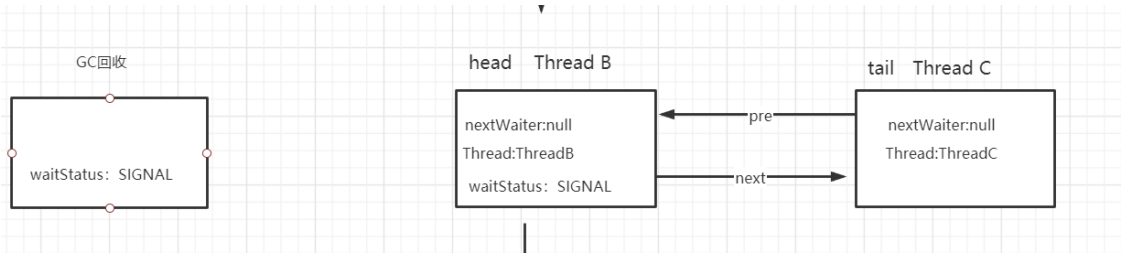
当 A 线程 unlock 释放锁后，线程 B 的线程被唤醒，从 park 的时候继续开始执行，阻塞在 acquireQueued 方法

```
final boolean acquireQueued(final Node node, int arg) {
    boolean failed = true;
    try {
        boolean interrupted = false;
        for (;;) {
            //threadA 解锁后，继续执行，拿到head 节点
            final Node p = node.predecessor();
            //p==head 节点，并且threadB 去尝试加锁，这个时候A 已经释放锁，进
入该逻辑
            if (p == head && tryAcquire(arg)) {
                setHead(node);
                //将B 节点设置为head 节点，里面逻辑同时把node 的前指针以及t
hread 设为null
                p.next = null; // help GC  //将p 的next 执行去除

                failed = false;
                return interrupted;
            }
            if (shouldParkAfterFailedAcquire(p, node) &&
                parkAndCheckInterrupt())
                //当线程A 解锁后，ThreaB 从这里开始运行，继续for 循环

                interrupted = true;
        }
    } finally {
        if (failed)
            cancelAcquire(node);
    }
}
```

继续执行 for 循环，进入第一个 if 后，链表的节点变成如下



7.3 为什么是非公平锁

因为我任意一个新的线程都会在之前加锁的线程释放锁的时候抢占到锁，即在 unparkSuccessor 方法中，A 释放锁，但是还没有调用 LockSupport.unpark(s.thread)方法的时候，有新的线程去调用了 lock，并且加锁成功，所以，非公平就体现出来了，我后来的新线程可以在我之前的线程抢先获得锁

那么，公平锁又是怎么实现的，我们知道默认是非公平锁，那么我们看下怎么做到公平

7.4 公平锁怎么实现

公平锁，拿到锁后，直接调用

```
public final void acquire(int arg) {
    if (!tryAcquire(arg) &&
        acquireQueued(addWaiter(Node.EXCLUSIVE), arg))
        selfInterrupt();
}
```

然后调用 tryAcquire 方法

```
protected final boolean tryAcquire(int acquires) {
    //得到当前线程
    final Thread current = Thread.currentThread();
    int c = getState();
    //能抢占到锁，假如A 释放了，我们新进来一个线程去抢占锁
    if (c == 0) {
        if (!hasQueuedPredecessors() &&
            compareAndSetState(0, acquires)) {
            setExclusiveOwnerThread(current);
            return true;
        }
    }
    else if (current == getExclusiveOwnerThread()) {
        int nextc = c + acquires;
```

```
        if (nextc < 0)
            throw new Error("Maximum lock count exceeded");
        setState(nextc);
        return true;
    }
    return false;
}
```

我们发现公平锁与非公平锁多了一个 hasQueuedPredecessors 方法判断，只有当 hasQueuedPredecessors 返回 false 的时候，我们才会去尝试加锁

```
public final boolean hasQueuedPredecessors() {
    // The correctness of this depends on head being initialized
    // before tail and on head.next being accurate if the current
    // thread is first in queue.
    Node t = tail; //首先读取tail，为了避免tail 不为空，head 为空的情况，初始化逻辑
    //if (compareAndSetHead(new Node()))    //tail = head;
    Node h = head;
    Node s;
    return h != t &&
        ((s = h.next) == null || s.thread != Thread.currentThread());
}
```

我们来分析下，

1. h != t 就是我的队列里面不止一个 node 节点，就是有 node 节点在等待，那么返回 true,接下来去判断&&后面的逻辑,如何后面的逻辑也为 true，就不让抢占，但是如果后面的为 false，就能抢占
    1. 如果 h==t，有 2 种情况，第一种 2 个都为 null,说明链表还没有初始化，没有待执行线程，第二种都不为空，但是相等 说明链表已经只有一个线程在执行并且执行完了没有情况，新来的线程可以去拿到
2. ((s = h.next) == null || s.thread != Thread.currentThread()); 这里因为第一个条件为 true 了，所以这个条件必须为 false 才能够去抢占锁，那么 s = h.next) == null 和 s.thread != Thread.currentThread()必须都为 false
    1. (s = h.next) == null 表示有一个线程已经在抢占了，但是还没有完全入链表，不可抢占
    2. s.thread != Thread.currentThread() 下一个要执行的 node 的 thread 不是当前的线程，不可抢占

~~CLH(Craig, Landin, and Hagersten locks):~~

帅气的景天

以上就能够保证，我要抢占锁，必须按照顺序来，FIFO，也就保证了线程的公平性

7.5 状态

SIGNAL -1 必须后续需要去 unpark

CANCELLED 1 取消状态

CONDITION -2 等待条件状态,在等待队列中

PROPAGATE -3 状态需要向后传播

7.6 interrupted 的作用

除了 unpark 外，还有 interrupt 可以中断，该作用就是当 interrupt 中断时，我能捕捉到，传递我的线程是否有被 interrupt 中断过

（见例子），如果有中断过，并且拿到锁，执行 selfInterrupt，告诉外面我这个线程被 interrupt 中断过

7.7park 和 unpark

1）不会抛中断异常

2）使用更加灵活（相对于 wait 和 notify，它们必须在加锁的代码块内），可以在任何地方用

3）可以指定锁和解锁哪个线程

帅气的景天